PROJECT ARCHITECTURE

AI/ML-Based Campus Maintenance Reporting System

Capstone Design Project | Team INNOVEXA

AI & Computer Engineering Department | 2026 Second Semester

Team: ISLAM MD KAYES • ZEHAD AMZAD BIN MIR • PRANTO MD TAIF RAHMAN

Deliverables: UML Use Case, Activity Workflow, Sequence Workflow, Application Wireframes, ER Database Structure, System Architecture and Technology Stack.

Note: Screens and values are illustrative design examples, not actual system results.

1. UML Use Case Overview

Actor	Main use cases
Student	Login; submit facility report; upload photo; enter location; track status; receive updates; provide feedback.
Maintenance staff	View/accept assigned tasks; update progress; add repair notes; upload completion evidence; mark resolved.
Administrator	Monitor reports; manage departments; review escalations; monitor workload; view analytics and recurring problems.
AI/ML service	Classify photo and text; detect possible duplicate reports; identify supported hazards; estimate severity; predict priority; recommend department.
Notification/escalation service	Send updates; monitor response targets; send reminders; alert administrators about overdue urgent tasks.

2. UML Activity Workflow

Step	Activity
1	Student identifies a facility issue.
2	Student submits photo, description, and building/floor/room.
3	Backend validates input and creates a report.
4	Multimodal AI/ML classifies image and text.
5	Duplicate detection compares active reports in the same/nearby location.
6	Computer vision checks supported hazard classes; severity/safety risk is estimated.
7	ML model estimates priority using available report and historical features.
8	Routing logic recommends Electrical, Plumbing, HVAC, IT, or another department.
9	Backend creates a maintenance task and notifies staff.
10	Staff accepts task, updates progress, and records repair details.
11	Backend stores status changes and notifies the student.
12	Response monitor checks the task's response target.
13	If overdue, reminders are sent and urgent unresolved tasks are escalated.
14	Admin dashboard summarizes workload, resolution time, and recurring issues.

3. UML Sequence Workflow

Order	Student/UI	Backend	AI/ML	Database	Staff/Admin
1	Submit photo, text, location	Validate	—	Create report	—
2	Wait	Request analysis	Classify; duplicate check; hazard/severity assessment	—	—
3	Show status	Combine analysis with routing/priority	Return predictions/confidence	Save analysis/task	Receive task
4	—	Notify staff	—	Save notification	View/accept
5	View status	Process status change	—	Save history	Update/resolve
6	Receive update	Check response target	—	Read timestamps	—
7	—	Escalate if overdue	—	Save escalation	Admin reviews
8	—	Query metrics	—	Return aggregates	View analytics

4. Application Mockups / Wireframes

Low-fidelity examples of the planned screens. Sample values are illustrative.

A. Student dashboard

```
CAMPUS MAINTENANCE [Bell] [Profile]
Welcome, Student
[ + REPORT A PROBLEM ]
MY REPORTS
Broken AC | Building A / Room 301 | IN PROGRESS
Broken light | Building B / Room 204 | RESOLVED
```

B. Submit a report

```
REPORT CAMPUS ISSUE
Photo: [ Upload image ]
Building: [ Engineering Building v ]
Floor: [ 3 v ] Room: [ 301 ]
Description: [ Water leaking near AC... ]
[ Submit report ]
```

C. AI analysis and routing

```
REPORT ANALYSIS (ILLUSTRATIVE)
Category: HVAC / Water leakage
Possible duplicate: Review result
Hazard/severity: Flag for staff verification
Priority: HIGH (example)
Suggested department: HVAC / Plumbing
[ Confirm recommendation ] [ Request review ]
```

D. Maintenance dashboard

```
MAINTENANCE TASKS
[CRITICAL] Exposed wiring — Building A
Safety flag: HIGH | [Accept] [View report]
[MEDIUM] Classroom light — Building B
Status: Assigned | [Update status]
```

E. Administrator dashboard

```
ADMIN DASHBOARD
Active | High-priority | Resolved | Overdue
Workload: Electrical | HVAC | Plumbing | IT
Charts: reports by category / building / month
Recurring issues | Overdue escalation queue
```

5. ER Diagram / Database Structure

Entity	Primary key	Important fields / relationships
USERS	user_id	name, student_id, email, role
BUILDINGS	building_id	name, description
ROOMS	room_id	building_id → BUILDINGS; floor, room_number
REPORTS	report_id	user_id → USERS; room_id → ROOMS; description, image_path, category, status, created_at
AI_ANALYSIS	analysis_id	report_id → REPORTS; prediction, confidence, duplicate_score, hazard_flags, severity, priority, suggested_department_id
DEPARTMENTS	department_id	name, description
MAINTENANCE_TASKS	task_id	report_id → REPORTS; department_id → DEPARTMENTS; staff_id → USERS; priority, status, timestamps
STATUS_HISTORY	history_id	report_id → REPORTS; changed_by → USERS; old/new status, note, changed_at
NOTIFICATIONS	notification_id	user_id → USERS; report_id → REPORTS; message, is_read, created_at
FEEDBACK	feedback_id	report_id → REPORTS; user_id → USERS; rating, comment, created_at

Relationship summary

Relationship	Cardinality
USERS → REPORTS	One user submits many reports (1:N).
BUILDINGS → ROOMS	One building contains many rooms (1:N).
ROOMS → REPORTS	One room can have many reports (1:N).
REPORTS → AI_ANALYSIS	One report can have multiple analysis records (1:N).
REPORTS → MAINTENANCE_TASKS	One report can create multiple tasks if work is split (1:N).
DEPARTMENTS → MAINTENANCE_TASKS	One department receives many tasks (1:N).
USERS → STATUS_HISTORY	One user can make many status changes (1:N).
USERS → NOTIFICATIONS	One user can receive many notifications (1:N).
REPORTS → FEEDBACK	A report can receive zero or one final feedback entry (1:0..1).

6. System Architecture and Technology Stack

Layer	Technology / responsibility
Frontend	React.js — student reporting, status tracking, staff portal, administrator dashboard.
Backend API	Python + FastAPI — authentication, report validation, workflow, routing, notifications, escalation.
Database	PostgreSQL — users, locations, reports, AI outputs, tasks, history, notifications, feedback.
AI/ML	Python — image/text classification, duplicate similarity, supported hazard detection, severity assessment, priority prediction.
Analytics	Pandas / SQL — reports by category, building, department, resolution time, recurring issues.
Testing	API/workflow tests, usability checks, model evaluation with suitable labelled data.

High-level data flow

Stage	Description
1. User interface	Student submits photo, text, and campus location.
2. Backend	Validates input and creates report.
3. AI/ML	Returns category, duplicate likelihood, supported hazard flags, severity and priority estimates.
4. Dispatch	Backend recommends department and creates task for review/assignment.
5. Maintenance	Staff accepts task and records progress/resolution.
6. Notifications	Student receives updates; overdue urgent tasks can be escalated.
7. Analytics	Administrator reviews workload, resolution performance, recurring issues.

Scope: QR codes are not required. Reports use photos, text, and campus location. Hazard detection is limited to classes supported by the chosen model and dataset. Critical cases require staff review; AI assists decisions and does not replace professional inspection.

Suggested implementation order: Requirements → database schema → backend reporting → React screens → AI/ML baseline → classification and duplicate detection → hazard/severity → priority/routing → notifications/escalation → analytics → integration/testing.